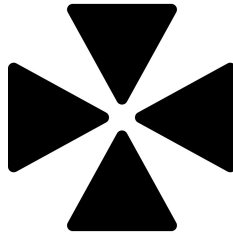


**Otomatisasi Pembuatan Dokumen Laporan Akademik Menggunakan
Pandoc, LaTeX, dan Markdown
Laporan Dokumentasi Pipeline**

Nama Mata Kuliah

Dosen Pengampu:

Nama Dosen Pengampu



Disusun oleh:

Nama Mahasiswa 1 101234567

Nama Mahasiswa 2 101234568

Nama Mahasiswa 3 101234569

**Program Studi Informatika
Universitas Example
2026/2027**

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga laporan yang berjudul **“Otomatisasi Pembuatan Dokumen Laporan Akademik menggunakan Pandoc, LaTeX, dan Markdown”** dapat diselesaikan dengan baik.

Laporan ini disusun sebagai dokumentasi pipeline otomatisasi dokumen akademik yang dibangun menggunakan kombinasi teknologi Markdown, LaTeX, Pandoc, ImageMagick, dan Bash. Pipeline ini memungkinkan penulisan konten laporan dalam format Markdown yang sederhana, yang kemudian dikonversi menjadi PDF dengan format profesional melalui Pandoc dan pdfLaTeX.

Penulis menyadari bahwa laporan ini tidak dapat terselesaikan tanpa bantuan dari berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih kepada semua pihak yang telah memberikan dukungan dalam penyelesaian laporan ini.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan untuk perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat bagi pembaca dalam memahami konsep otomatisasi dokumen akademik.

Juli 2026

Tim Penyusun

DAFTAR ISI

BAB I	PENDAHULUAN	1
1.1	Latar Belakang	1
1.2	Rumusan Masalah	2
1.3	Tujuan	2
1.4	Manfaat	2
1.5	Batasan.....	3
BAB II	TINJAUAN PUSTAKA	4
2.1	Markdown	4
2.1.1	Sejarah dan Perkembangan	4
2.1.2	Sintaks Dasar	4
2.1.3	Keunggulan untuk Laporan Akademik	5
2.2	LaTeX.....	5
2.2.1	Sejarah dan Perkembangan	5
2.2.2	Struktur Dokumen LaTeX.....	6
2.2.3	Packages Penting	6
2.2.4	Font dan Encoding	7
2.2.5	Pengaturan Halaman dan Margin	7
2.2.6	Penomoran dan Daftar Isi	7
2.3	Pandoc	8
2.3.1	Sejarah dan Perkembangan	8
2.3.2	Arsitektur Pandoc	8
2.3.3	Format Input dan Output	8
2.3.4	Flags Penting	9
2.3.5	Template Engine	9
2.4	ImageMagick	9
2.4.1	Sejarah dan Perkembangan	9
2.4.2	Perintah Convert	10
2.4.3	Penanganan Alpha Channel	10
2.4.4	Optimasi Gambar	10
2.5	Bash dan Linux	10
2.5.1	Bash Shell	10
2.5.2	Konsep Dasar Shell Scripting	11
2.5.3	Fitur Penting dalam Pipeline Script	11
2.5.4	Linux sebagai Platform Pipeline	12

BAB III	METODOLOGI	13
3.1	Arsitektur Pipeline	13
3.1.1	Tahap Input	13
3.1.2	Tahap Proses	14
3.1.3	Tahap Output	14
3.2	Template LaTeX	14
3.2.1	Struktur Template	14
3.2.2	Variabel Pandoc	16
3.3	Build Script	16
3.3.1	Alur Kerja Build Script	16
3.3.2	Penanganan Error	17
3.4	Penulisan Konten Markdown	18
3.4.1	Struktur File	18
3.4.2	Penulisan Tabel	18
3.4.3	Penulisan Matematika	18
3.4.4	Penulisan Kode	19
3.5	Studi Kasus: Project Laporan Generator	19
BAB IV	HASIL DAN PEMBAHASAN	20
4.1	Struktur Direktori	20
4.2	Analisis Template LaTeX	21
4.2.1	Document Class dan Font	21
4.2.2	Margin dan Spasi	21
4.2.3	Penomoran Bab dan Section	21
4.2.4	Format Chapter	22
4.3	Analisis Build Script	22
4.3.1	Setup dan Error Handling	22
4.3.2	Proses Copy dan Preparasi	22
4.3.3	Eksekusi Pandoc	23
4.3.4	Font Definition Generation	23
4.4	Output Pipeline	23
4.4.1	File yang Dihasilkan	23
4.4.2	Perbandingan dengan Metode Konvensional	24
4.5	Version Control dengan Git	24
4.5.1	Keunggulan File Teks	24
4.5.2	Workflow Kolaborasi	25
4.5.3	.gitignore	25
BAB V	PENUTUP	26
5.1	Kesimpulan	26

5.2	Saran	26
DAFTAR PUSTAKA		28

BAB I

PENDAHULUAN

1.1 Latar Belakang

Penulisan laporan akademik merupakan kegiatan yang tidak terpisahkan dari dunia pendidikan. Mahasiswa, dosen, dan peneliti secara rutin menghasilkan laporan tugas akhir, skripsi, tesis, makalah, jurnal, dan berbagai dokumen akademik lainnya. Kualitas laporan tidak hanya diukur dari isi ilmiahnya, tetapi juga dari format penyajian dan konsistensi tampilan.

Metode penulisan laporan yang umum digunakan saat ini masih mengandalkan word processor seperti Microsoft Word atau LibreOffice Writer. Pendekatan ini memiliki beberapa kelemahan signifikan:

1. **Inkonsistensi format** — Setiap pengguna memiliki cara berbeda dalam mengatur format. Perubahan template di tengah pengerjaan seringkali membutuhkan penyesuaian manual satu per satu pada setiap bagian dokumen.
2. **Revisi yang memakan waktu** — Perubahan format, penomoran, atau tata letak seringkali mengharuskan pengguna mengedit ulang seluruh dokumen secara manual. Hal ini sangat tidak efisien, terutama untuk dokumen panjang.
3. **Ketidakkcocokan dengan version control** — Format file word processor bersifat binary (bukan teks biasa), sehingga tidak dapat di-diff dengan mudah. Git dan sistem version control lainnya tidak dapat melacak perubahan secara granular pada file seperti .docx.
4. **Kolaborasi yang rumit** — Menggabungkan perubahan dari beberapa kontributor seringkali menyebabkan masalah format, layout yang berantakan, atau bahkan data hilang.

Konsep pemisahan konten dari tampilan (separation of concerns) telah lama dikenal dalam dunia pengembangan perangkat lunak. Dalam konteks penulisan dokumen, konsep ini berarti bahwa penulis cukup fokus pada konten laporan, sementara tampilan dan format ditangani secara terpisah oleh sistem typesetting.

Pendekatan yang ditawarkan adalah kombinasi tiga teknologi utama: - **Markdown** sebagai format penulisan konten yang ringan dan mudah dibaca - **LaTeX** sebagai sistem typesetting untuk format dokumen profesional - **Pandoc** sebagai universal document converter yang menjembatani Markdown dan LaTeX

Ketiga teknologi ini digabungkan dalam sebuah pipeline otomatisasi yang dijalankan melalui skrip Bash. Pipeline ini mengambil file Markdown sebagai input, memprosesnya dengan

Pandoc menggunakan template LaTeX, dan menghasilkan output PDF yang siap digunakan. Seluruh proses dijalankan dengan satu perintah di terminal.

Laporan ini membahas secara rinci pipeline otomatisasi dokumen akademik tersebut, mulai dari teknologi yang digunakan, arsitektur pipeline, implementasi, hingga studi kasus penerapannya pada project Laporan Generator.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam laporan ini adalah:

1. Apa yang dimaksud dengan Markdown, LaTeX, dan Pandoc, serta bagaimana peran masing-masing dalam otomatisasi dokumen?
2. Bagaimana struktur template LaTeX yang digunakan untuk menghasilkan laporan akademik?
3. Bagaimana alur kerja konversi dokumen dari format Markdown ke PDF menggunakan Pandoc?
4. Bagaimana cara menangani gambar dalam pipeline otomatisasi dokumen?
5. Bagaimana skrip Bash digunakan untuk mengotomatisasi seluruh proses build?
6. Bagaimana struktur file dan implementasi pipeline pada project Laporan Generator?

1.3 Tujuan

Tujuan dari laporan ini adalah:

1. Menjelaskan konsep Markdown, LaTeX, dan Pandoc serta peran masing-masing dalam pipeline otomatisasi dokumen
2. Menguraikan struktur template LaTeX yang digunakan untuk laporan akademik
3. Menjelaskan alur konversi dokumen dari Markdown ke PDF menggunakan Pandoc dengan engine pdfLaTeX
4. Menjelaskan teknik penanganan gambar dalam pipeline menggunakan ImageMagick
5. Menjelaskan implementasi skrip Bash sebagai orkestrator pipeline
6. Menyajikan studi kasus implementasi pipeline pada project Laporan Generator, termasuk struktur file dan cara penggunaannya

1.4 Manfaat

Manfaat dari laporan ini adalah:

1. Memberikan pemahaman tentang konsep pemisahan konten dan tampilan dalam penulisan dokumen akademik

2. Menyediakan referensi implementasi pipeline otomatisasi dokumen yang dapat direproduksi
3. Meningkatkan efisiensi pembuatan laporan akademik melalui otomatisasi
4. Menghasilkan format laporan yang konsisten, profesional, dan sesuai standar
5. Memudahkan kolaborasi dan version control melalui penggunaan format teks

1.5 Batasan

Batasan dalam laporan ini adalah:

1. Output dokumen yang dihasilkan dalam format PDF
2. Tools utama yang digunakan: Pandoc, pdfLaTeX, ImageMagick, dan Bash
3. Template LaTeX menggunakan font Nimbus Serif sebagai pengganti Times New Roman
4. Ukuran kertas A4 dengan margin kiri dan kanan 2,5 cm, atas 2 cm, dan bawah 3 cm
5. Sistem operasi yang digunakan adalah Linux (Ubuntu/Debian), meskipun konsep yang diterapkan berlaku universal

BAB II

TINJAUAN PUSTAKA

2.1 Markdown

2.1.1 Sejarah dan Perkembangan

Markdown diciptakan oleh John Gruber pada tahun 2004 dengan tujuan menciptakan format penulisan yang mudah dibaca dan mudah ditulis dalam bentuk teks biasa (Gruber, 2004). Filosofi utama Markdown adalah bahwa format teks biasa sudah cukup untuk menulis dokumen sederhana, dan elemen markup yang digunakan harus seminimal mungkin.

Sejak diperkenalkan, Markdown telah berkembang menjadi berbagai varian dan ekstensi. Beberapa varian populer antara lain GitHub Flavored Markdown (GFM), CommonMark, dan Pandoc Markdown. Masing-masing varian memiliki fitur tambahan seperti dukungan tabel, daftar tugas (task list), sintaks matematika, dan metadata.

Pandoc Markdown adalah varian yang paling kaya fitur karena dikembangkan khusus untuk Pandoc. Varian ini mendukung hampir semua ekstensi yang ada, termasuk tabel multi-baris, catatan kaki, definisi metadata (YAML front matter), sintaks matematika LaTeX, dan berbagai jenis blok kode.

2.1.2 Sintaks Dasar

Berikut adalah sintaks dasar Markdown yang digunakan dalam penulisan laporan:

Elemen	Sintaks	Contoh
Heading level 1	# Teks	# BAB 1: PENDAHULUAN
Heading level 2	## Teks	## 1.1 Latar Belakang
Heading level 3	### Teks	### 2.1.1 Sejarah
Paragraf	Baris teks biasa, dipisahkan baris kosong	
Teks tebal	**teks** atau __teks__	**penting**
Teks miring	<i>*teks*</i> atau <i>_teks_</i>	<i>*catatan*</i>
List tidak berurut	- item atau * item	- poin pertama
List berurut	1. item	1. langkah pertama
Link	[teks] (url)	[Pandoc] (https://pandoc.org)
Gambar	![caption] (path/file.png)	![Diagram] (gambar/arsitektur.png)
Tabel	Menggunakan pipe dan dash	(lihat contoh di bawah)
Kode inline	`kode`	`print("hello")`
Blok kode	Diapit triple backtick	

Elemen	Sintaks	Contoh
Matematika inline	$\$ \dots \$$	$E = mc^2$
Matematika display	$\$ \$ \dots \$ \$$	$\sum_{i=1}^n i$

2.1.3 Keunggulan untuk Laporan Akademik

Markdown memiliki beberapa keunggulan yang membuatnya cocok sebagai format penulisan laporan akademik:

1. **Format teks biasa** — File Markdown dapat dibaca dan diedit dengan editor teks apa pun. Tidak memerlukan perangkat lunak berbayar atau aplikasi khusus.
2. **Kompatibilitas dengan Git** — Karena berbasis teks, file Markdown dapat di-version control dengan Git. Perubahan dapat di-diff, di-merge, dan dilacak dengan mudah.
3. **Sintaks sederhana** — Markdown dapat dipelajari dalam hitungan menit. Penulis dapat langsung fokus pada konten tanpa terganggu oleh kompleksitas format.
4. **Konversi multi-format** — File Markdown dapat dikonversi ke berbagai format output seperti PDF, HTML, EPUB, DOCX, dan LaTeX.
5. **Integrasi LaTeX** — Markdown mendukung sintaks matematika LaTeX secara inline maupun display, memungkinkan penulisan rumus ilmiah dengan mudah.
6. **Portabilitas** — File Markdown bersifat platform-independen dan dapat dibuka di sistem operasi apa pun.

2.2 LaTeX

2.2.1 Sejarah dan Perkembangan

LaTeX dikembangkan oleh Leslie Lamport pada tahun 1994 sebagai kumpulan makro (macro package) di atas sistem typesetting TeX yang diciptakan oleh Donald Knuth pada tahun 1984 (Knuth, 1984; Lamport, 1994). LaTeX menyediakan antarmuka yang lebih mudah digunakan dibandingkan TeX murni, dengan menyediakan perintah-perintah standar untuk format dokumen.

LaTeX dirancang berdasarkan prinsip bahwa penulis harus fokus pada konten dan struktur logis dokumen, bukan pada tampilan visual. Format dan tata letak ditangani secara otomatis oleh sistem berdasarkan kelas dokumen dan packages yang digunakan.

2.2.2 Struktur Dokumen LaTeX

Dokumen LaTeX terdiri dari dua bagian utama:

1. **Preamble** — Bagian sebelum `\begin{document}` yang berisi:

Deklarasi document class: `\documentclass[12pt,a4paper,oneside]{book}`

Penggunaan packages: `\usepackage{geometry}`, `\usepackage{setspace}`, dll.

Pengaturan global: font, margin, header, footer

Definisi perintah kustom

2. **Body** — Bagian antara `\begin{document}` dan `\end{document}` yang berisi konten dokumen:

Front matter: cover, kata pengantar, daftar isi

Main matter: bab-bab utama

Back matter: daftar pustaka, lampiran

Document class yang digunakan dalam pipeline ini adalah book. Kelas ini menyediakan struktur chapter, section, dan subsection yang sesuai untuk laporan akademik. Pengaturan 12pt menentukan ukuran font dasar, a4paper menentukan ukuran kertas, dan oneside mengatur pencetakan satu sisi.

2.2.3 Packages Penting

Berikut adalah packages yang digunakan dalam template LaTeX:

Package	Fungsi
geometry	Pengaturan margin dan ukuran kertas
setspace	Pengaturan spasi baris (1.5 spacing)
graphicx	Penyisipan gambar ke dalam dokumen
hyperref	Pembuatan tautan internal dan eksternal
fancyhdr	Header dan footer kustom
titlesec	Format judul chapter dan section
tocloft	Format daftar isi
indentfirst	Indentasi pada paragraf pertama setelah judul
caption	Format caption gambar dan tabel
float	Penempatan floating objects (gambar, tabel)
listings	Penulisan kode program dengan syntax highlighting
xcolor	Penggunaan warna dalam dokumen
enumitem	Format daftar (list) kustom
longtable	Tabel yang bisa melebihi satu halaman

Package	Fungsi
booktabs	Garis tabel profesional
amsmath	Persamaan matematika tingkat lanjut

2.2.4 Font dan Encoding

Font default yang digunakan adalah Nimbus Serif, yang merupakan font open source dengan bentuk hampir identik dengan Times New Roman. Nimbus Serif adalah bagian dari proyek font URW++ yang dirilis di bawah lisensi GPL.

Encoding font menggunakan T1 (Cork encoding) melalui package fontenc. T1 encoding mendukung karakter-karakter bahasa Indonesia dan Eropa Barat dengan lebih baik dibandingkan encoding default OT1. Font Nimbus Serif diaktifkan melalui file mapping `nimbus15.map` dan perintah `\renewcommand{\rmdefault}{NimbusSerif}`.

Definisi font untuk Nimbus Serif memerlukan file `fonttools_ts1.enc` untuk encoding TS1 (Text Companion Symbols) dan file `t1jtm.fd` untuk mendeklarasikan font shape keluarga JTM yang digunakan oleh Nimbus.

2.2.5 Pengaturan Halaman dan Margin

Pengaturan halaman dilakukan dengan package `geometry`:

Margin kiri: 2,5 cm
 Margin kanan: 2,5 cm
 Margin atas: 2 cm
 Margin bawah: 3 cm
 Jarak footer: 40pt dari bawah

Spasi baris diatur menjadi 1,5 spasi menggunakan package `setspace` dengan perintah `\onehalfspacing`. Indentasi paragraf diatur sebesar 1,5 cm dengan `\setlength{\parindent}{1.5cm}`.

2.2.6 Penomoran dan Daftar Isi

Penomoran bab menggunakan angka Romawi dengan prefix “BAB”: - Output: BAB I, BAB II, BAB III, dst. - Konfigurasi: `\renewcommand{\chaptername}{BAB}` dan `\renewcommand{\thechapter}{\Roman{chapter}}`

Penomoran sub-bab menggunakan angka Arab: - Output: 1.1, 1.2, 2.1, 2.2, dst. - Konfigurasi: `\renewcommand{\thesection}{\arabic{chapter}.\arabic{section}}`

Daftar isi menampilkan entri BAB dengan format tebal dan prefix “BAB”: - Konfigurasi: `\renewcommand{\cftchappresnum}{BAB }` - Spasi antar entri diatur dengan `\setlength{\cftbeforechapskip}{5pt}`

2.3 Pandoc

2.3.1 Sejarah dan Perkembangan

Pandoc dikembangkan oleh John MacFarlane sejak tahun 2006 (MacFarlane, 2024). Nama “Pandoc” berasal dari bahasa Yunani yang berarti “semua hal yang diterima”. Pandoc adalah universal document converter yang dapat mengkonversi dokumen antar berbagai format markup.

Awalnya dikembangkan untuk memenuhi kebutuhan konversi antara Haskell documentation dan Markdown, Pandoc kini telah berkembang menjadi salah satu alat konversi dokumen paling komprehensif yang tersedia. Pandoc ditulis dalam bahasa pemrograman Haskell dan dirilis di bawah lisensi GPL.

2.3.2 Arsitektur Pandoc

Pandoc bekerja melalui arsitektur berbasis **AST** (Abstract Syntax Tree). AST adalah representasi internal dokumen yang bersifat netral terhadap format input maupun output.

Proses konversi terdiri dari tiga tahap:

1. **Parsing (Reader)** — Pandoc membaca dokumen input dan mem-parsing-nya menjadi AST. Setiap format input memiliki reader yang sesuai (Markdown reader, HTML reader, LaTeX reader, dll.).
2. **Transformasi (AST)** — AST yang dihasilkan dapat ditransformasi atau difilter sebelum dikonversi. Filter dapat ditulis dalam Lua, Python, atau bahasa lain untuk memodifikasi AST.
3. **Generasi (Writer)** — Pandoc menulis AST ke format output yang diinginkan melalui writer yang sesuai (PDF writer, HTML writer, DOCX writer, dll.).

Pendekatan AST ini memungkinkan Pandoc untuk mengkonversi antar format yang sangat beragam dengan hasil yang konsisten. Jika suatu fitur didukung oleh AST, fitur tersebut secara otomatis dapat dikonversi ke format output apa pun.

2.3.3 Format Input dan Output

Pandoc mendukung berbagai format input dan output:

Format Input: - Markdown (beberapa varian) - HTML - LaTeX - Docx (Microsoft Word) - EPUB - OPML - org-mode (Emacs) - reStructuredText - Textile - MediaWiki markup - Jupyter Notebook (.ipynb)

Format Output: - PDF (melalui engine LaTeX) - HTML (+ slide show) - DOCX - LaTeX - EPUB - Markdown - AsciiDoc - Man page - Plain text

2.3.4 Flags Penting

Berikut adalah flags Pandoc yang digunakan dalam pipeline:

Flag	Fungsi	Contoh Penggunaan
-template=FILE	Menentukan template LaTeX	-template=template.latex
-include-before-body=FILE	Menyisipkan konten sebelum body	-include-before-body=cover.md
-include-after-body=FILE	Menyisipkan konten setelah body	-include-after-body=daftar-pustaka.md
-top-level-division=TYPE	Menentukan tipe division tertinggi	-top-level-division=chapter
-pdf-engine=ENGINE	Menentukan engine PDF	-pdf-engine=pdflatex
-o FILE	Nama file output	-o Laporan.pdf
-from=FORMAT	Format input (auto-detect)	-from=markdown
-to=FORMAT	Format output (auto-detect)	-to=latex

2.3.5 Template Engine

Pandoc menggunakan sistem template yang fleksibel. Template ditulis dalam format yang mirip dengan format output, dengan tambahan variabel yang diisi oleh Pandoc.

Variabel template yang umum digunakan: - `$body$` — Konten utama dokumen - `$title$` — Judul dokumen - `$author$` — Penulis - `$date$` — Tanggal - `$for(include-before)$``include-before``$endfor$` — Konten sebelum body - `$for(include-after)$``include-after``$endfor$` — Konten setelah body - `toc` — Daftar isi - `$header-includes$` — Kustomisasi header LaTeX

Template memungkinkan kustomisasi penuh terhadap format output tanpa mengubah konten dokumen. Ini adalah inti dari konsep pemisahan konten dan tampilan.

2.4 ImageMagick

2.4.1 Sejarah dan Perkembangan

ImageMagick adalah software suite untuk manipulasi gambar yang pertama kali dikembangkan oleh John Cristy pada tahun 1990 (LLC, 2024). ImageMagick mendukung lebih dari 200 format file gambar dan menyediakan berbagai tool untuk mengolah gambar melalui command line.

ImageMagick ditulis dalam bahasa C dan dirilis di bawah lisensi Apache 2.0. Software ini tersedia untuk Linux, macOS, dan Windows.

2.4.2 Perintah Convert

Perintah `convert` adalah tool utama ImageMagick yang digunakan untuk: - Konversi antar format gambar (PNG ke JPG, dll.) - Resize gambar - Rotasi dan flipping - Manipulasi warna dan channel - Penambahan efek dan filter - Optimasi ukuran file

Dalam pipeline ini, `convert` digunakan terutama untuk menghapus alpha channel dari gambar PNG.

2.4.3 Penanganan Alpha Channel

Gambar PNG seringkali memiliki alpha channel yang merepresentasikan transparansi. Saat gambar dengan alpha channel dikonversi ke PDF oleh pdfLaTeX, area transparan dapat menyebabkan masalah seperti:

- Background gambar menjadi hitam

- Gambar tidak muncul sama sekali

- Warna gambar berubah

Untuk mengatasi masalah ini, perintah berikut digunakan:

```
convert input.png -alpha off output.png
```

Flag `-alpha off` menghapus alpha channel dari gambar dan mengganti area transparan dengan latar belakang putih. Gambar yang dihasilkan kompatibel penuh dengan pdfLaTeX.

2.4.4 Optimasi Gambar

Selain penanganan alpha channel, ImageMagick juga digunakan untuk mengoptimasi ukuran file gambar:

```
convert input.png -quality 85% -resize 800x600 output.jpg
```

Parameter `-quality` mengontrol tingkat kompresi (semakin rendah, semakin kecil ukuran file). Parameter `-resize` mengubah dimensi gambar. Kedua parameter ini membantu mengurangi ukuran file PDF akhir.

2.5 Bash dan Linux

2.5.1 Bash Shell

Bash (Bourne Again SHell) adalah shell dan bahasa scripting yang dikembangkan oleh Brian Fox pada tahun 1989 untuk GNU Project (Foundation, 2024). Bash adalah shell default pada sebagian besar distribusi Linux dan macOS.

Bash menggabungkan fitur-fitur dari berbagai shell sebelumnya: - Dari Bourne Shell (sh): syntax dasar, variabel, redirection - Dari C Shell (csh): history, job control, aliases - Dari Korn Shell (ksh): command completion, arithmetic

2.5.2 Konsep Dasar Shell Scripting

Shebang: Baris pertama script Bash dimulai dengan `#!/usr/bin/env bash` yang memberitahu sistem bahwa script harus dijalankan dengan interpreter Bash.

Variabel:

```
NAMA="Laporan Generator"
echo $NAMA
```

Command Substitution:

```
DIR="$(cd "$(dirname "$0")" && pwd)"
```

Perintah di atas mendapatkan direktori tempat script berada.

Conditional:

```
if [ -d "$OUTDIR/gambar" ]; then
    cp -r "$OUTDIR/gambar" "$TMPDIR/"
fi
```

Error Handling:

```
set -e
```

Perintah `set -e` menghentikan script jika ada perintah yang mengembalikan exit code non-zero (gagal). Ini mencegah script melanjutkan eksekusi setelah terjadi error.

2.5.3 Fitur Penting dalam Pipeline Script

`set -e`: Menghentikan eksekusi script jika ada perintah yang gagal. Ini penting untuk pipeline karena kegagalan pada salah satu tahap harus menghentikan seluruh proses.

`mktemp -d`: Membuat direktori temporer dengan nama unik. Direktori ini digunakan untuk menyimpan file-file sementara selama proses build.

```
TMPDIR=$(mktemp -d)
```

trap: Mendaftarkan perintah yang akan dijalankan saat script selesai atau menerima sinyal. Ini digunakan untuk membersihkan direktori temporer.

```
trap "rm -rf $TMPDIR" EXIT
```

Dengan `trap`, direktori temporer akan dihapus otomatis baik saat script selesai normal maupun saat terjadi error.

Exit Codes: Setiap perintah di Linux/Bash mengembalikan exit code: - 0: sukses - Non-0: gagal (angka berbeda menunjukkan jenis error berbeda)

Pipes dan Redirection:


```
pandoc ... 2>&1
```

2>&1 menggabungkan stderr (file descriptor 2) ke stdout (file descriptor 1), sehingga error dan output normal tercetak bersamaan.

2.5.4 Linux sebagai Platform Pipeline

Linux menyediakan lingkungan yang ideal untuk pipeline otomatisasi dokumen (Love, 2010; Welsh et al., 2019):

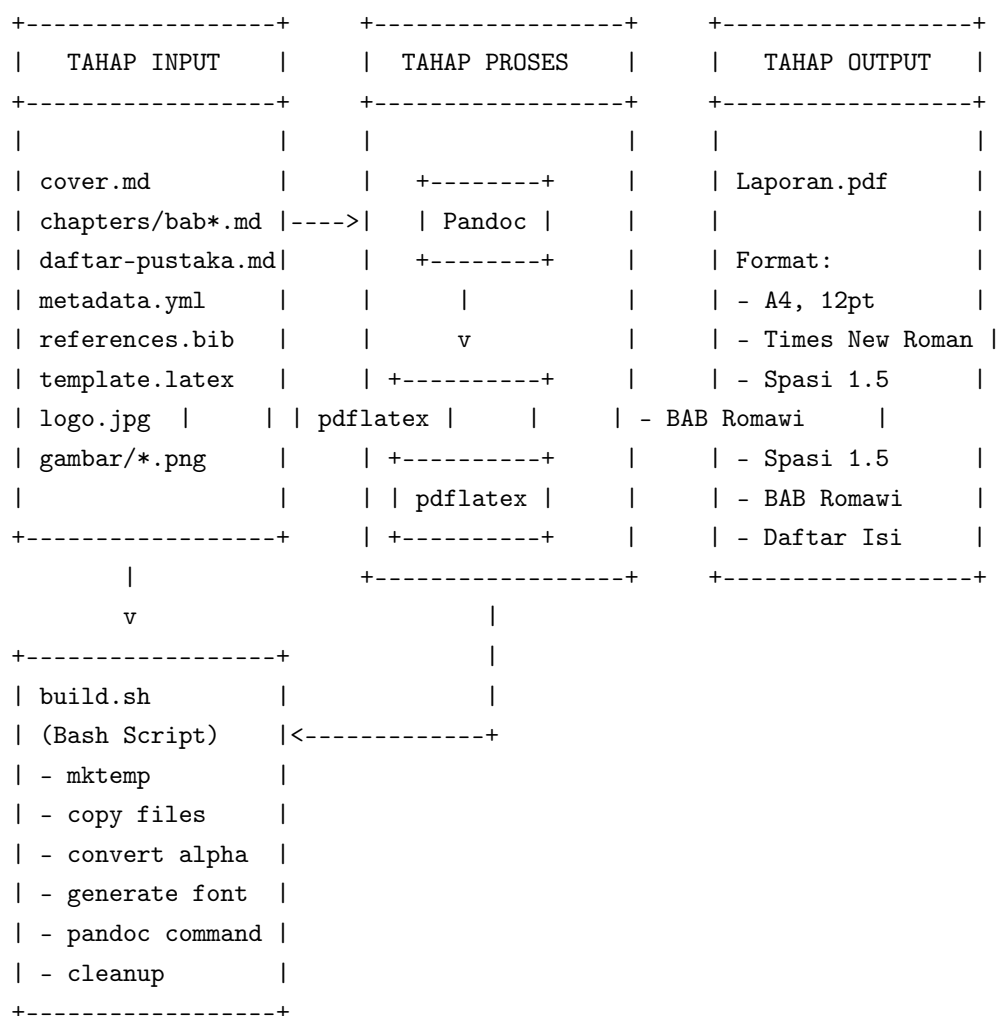
1. **Package Manager** — Distribusi Linux menyediakan package manager (apt, dnf, pacman) yang memudahkan instalasi Pandoc, TeX Live, dan ImageMagick.
2. **Filesystem** — Struktur filesystem Linux yang terstandarisasi memudahkan pengelolaan file dan path.
3. **Dukungan Bash** — Bash tersedia secara default dan terintegrasi penuh dengan sistem operasi.
4. **Headless Operation** — Pipeline dapat dijalankan di server tanpa GUI, memungkinkan integrasi dengan CI/CD.
5. **Skripability** — Linux menyediakan berbagai tool command line yang dapat digabungkan dalam pipeline scripting.

BAB III

METODOLOGI

3.1 Arsitektur Pipeline

Pipeline otomatisasi dokumen terdiri dari tiga tahap utama yang berjalan secara sekuensial:



3.1.1 Tahap Input

Tahap input terdiri dari persiapan file-file sumber yang diperlukan:

1. **File konten:** cover.md (halaman sampul), chapters/bab*.md (BAB 1-5), daftar-pustaka.md (referensi)

2. **File metadata:** `metadata.yml` (judul, penulis, institusi), `references.bib` (daftar pustaka)
3. **File template:** `template.latex` (format dan layout)
4. **File pendukung:** `logo.jpg` (logo institusi), `gambar/*.png` (gambar pendukung)

Semua file sumber ditulis dalam format Markdown (kecuali template yang menggunakan LaTeX dan logo yang menggunakan JPG).

3.1.2 Tahap Proses

Tahap proses dijalankan oleh skrip `build.sh` yang mengorkestrasi seluruh pipeline:

1. **Persiapan:** Membuat direktori temporer, menyalin file-file sumber
2. **Pre-processing:** Menghapus alpha channel dari gambar PNG
3. **Konversi:** Pandoc mengkonversi Markdown ke LaTeX menggunakan template
4. **Kompilasi:** pdfLaTeX mengkompilasi LaTeX menjadi PDF
5. **Pembersihan:** Menghapus direktori temporer

3.1.3 Tahap Output

Tahap output menghasilkan file PDF yang siap digunakan. File PDF ini memiliki format yang konsisten:

Ukuran kertas A4
Font Times New Roman (Nimbus Serif) 12pt
Spasi baris 1,5
Margin sesuai standar
Penomoran bab dengan angka Romawi (BAB I, BAB II)
Penomoran sub-bab dengan angka Arab (1.1, 2.1)
Daftar isi otomatis

3.2 Template LaTeX

Template LaTeX adalah komponen kunci yang menentukan format dan tampilan akhir dokumen. Template ini berisi seluruh pengaturan format yang diperlukan untuk menghasilkan laporan akademik standar Indonesia.

3.2.1 Struktur Template

Template LaTeX (`template.latex`) dibagi menjadi dua bagian utama:

Preamble (baris 1–109):

Document class:

```
\documentclass[12pt,a4paper,oneside]{book}
```

Pengaturan font:

```
\usepackage[T1]{fontenc}
\pdfmapfile{+nimbus15.map}
\renewcommand{\rmdefault}{NimbusSerif}
\renewcommand{\familydefault}{\rmdefault}
```

Package geometry untuk margin:

```
\usepackage[margin=2.5cm,top=2cm,bottom=3cm,footskip=40pt]{geometry}
```

Pengaturan chapter dan section numbering:

```
\renewcommand{\chaptername}{BAB}
\renewcommand{\thechapter}{\Roman{chapter}}
\renewcommand{\thesection}{\arabic{chapter}.\arabic{section}}
\renewcommand{\thesubsection}{\arabic{chapter}.\arabic{section}.\%
\arabic{subsection}}
```

Format chapter (14pt bold centered):

```
\titleformat{\chapter}[display]
{\normalfont\bfseries\centering\fontsize{14}{18}\selectfont}
{\chaptername\thechapter}{10pt}{\fontsize{14}{18}\selectfont}
```

Body (baris 111–137):

Front matter dengan halaman Romawi:

```
\frontmatter
\pagenumbering{roman}
```

Penyisipan cover melalui variabel Pandoc:

```
$for(include-before)$
$include-before$
$endfor$
```

Daftar isi:

```
\tableofcontents
```

Main matter dengan halaman Arab:

```
\mainmatter
\pagenumbering{arabic}
```

Konten utama:

```
$body$
```

3.2.2 Variabel Pandoc

Template menggunakan variabel Pandoc yang diisi secara otomatis: - `$for(include-before)$` — Berisi file yang disertakan melalui `--include-before-body` - `$body$` — Berisi seluruh konten file input Markdown

3.3 Build Script

Skrip `build.sh` adalah otak dari pipeline. Skrip ini mengatur seluruh proses build dari awal hingga akhir.

3.3.1 Alur Kerja Build Script

Langkah 1: Setup

```
set -e
DIR="$(cd "$(dirname "$0")" && pwd)"
OUTDIR="$DIR"
REPORT="$OUTDIR/Laporan.pdf"
TMPDIR=$(mktemp -d)
trap "rm -rf $TMPDIR" EXIT
```

Script menentukan direktori kerja, membuat direktori temporer dengan `mktemp`, dan mendaftarkan pembersihan otomatis dengan `trap`.

Langkah 2: Copy File Sumber

```
cp "$OUTDIR/cover.md" "$TMPDIR/"
cp "$OUTDIR/template.latex" "$TMPDIR/"
cp "$OUTDIR/logo.jpg" "$TMPDIR/"
cp "$OUTDIR/metadata.yml" "$TMPDIR/"
cp "$OUTDIR/references.bib" "$TMPDIR/"

if [ -d "$OUTDIR/chapters" ]; then
    cp "$OUTDIR/chapters"/*.md "$TMPDIR/"
else
    cp "$OUTDIR/isi-laporan.md" "$TMPDIR/"
fi
```

Semua file sumber disalin ke direktori temporer untuk menjaga kebersihan direktori kerja.

Langkah 3: Proses Gambar

```

if [ -d "$OUTDIR/gambar" ]; then
  cp -r "$OUTDIR/gambar" "$TMPDIR/"
  for f in "$TMPDIR/gambar/*.png"; do
    [ -f "$f" ] && convert "$f" -alpha off "$f" 2>/dev/null || true
  done
fi

```

Jika folder gambar/ ada, script menyalin dan memproses setiap file PNG untuk menghapus alpha channel.

Langkah 4: Generate File Pendukung

Script menghasilkan file `fontools_ts1.enc` yang diperlukan oleh encoding TS1 LaTeX, dan file `t1jtm.fd` yang mendefinisikan font shape untuk Nimbus Serif. Kedua file ditulis langsung dari dalam script menggunakan heredoc.

Langkah 5: Eksekusi Pandoc

```

pandoc \
  $INPUT_FILES \
  --template="template.latex" \
  --include-before-body="cover.md" \
  --include-after-body="daftar-pustaka.md" \
  --metadata-file="metadata.yml" \
  --citeproc \
  --bibliography="references.bib" \
  --top-level-division=chapter \
  --pdf-engine=pdfelatex \
  -o "$REPORT" 2>&1

```

Pandoc menjalankan konversi dengan parameter: - Input: `chapters/bab*.md` (BAB 1-5) - Template: `template.latex` - Metadata: `metadata.yml` (judul, penulis, institusi) - Cover: disisipkan sebelum body - Daftar pustaka: digenerate otomatis dari `references.bib` via `--citeproc` - Division: `chapter` (heading level 1 = `\chapter`) - Engine: `pdfLaTeX` - Output: `Laporan.pdf`

Error dan output normal digabungkan dengan `2>&1` untuk memudahkan debugging.

3.3.2 Penanganan Error

Script menggunakan beberapa mekanisme penanganan error:

1. `set -e` — Menghentikan script jika ada perintah yang gagal
2. `|| true` — Pada perintah `convert`, error diabaikan jika gambar tidak memerlukan pemrosesan

3. `trap ... EXIT` — Membersihkan direktori temporer meskipun terjadi error
4. `[ -f "$f" ]` — Memeriksa keberadaan file sebelum memproses

3.4 Penulisan Konten Markdown

3.4.1 Struktur File

Konten laporan ditulis dalam file-file terpisah per bab di direktori `chapters/` (misal `chapters/bab1-pendahuluan.md`, `chapters/bab2-tinjauan-pustaka.md`, dst.). Format penulisan mengikuti aturan Pandoc Markdown:

Heading level 1 (#) digunakan untuk judul BAB
 Heading level 2 (##) digunakan untuk sub-bab
 Heading level 3 (###) digunakan untuk sub-sub-bab
 Paragraf dipisahkan dengan baris kosong
 Tabel menggunakan sintaks pipe

3.4.2 Penulisan Tabel

Tabel ditulis menggunakan format pipe dengan baris header dan separator:

```
| Kolom 1 | Kolom 2 | Kolom 3 |
|-----|:-----:|-----:|
| Kiri    | Tengah  | Kanan    |
| Teks    | Teks    | Teks     |
```

Alignment dikontrol dengan posisi titik dua (:) pada baris separator: - :--- — Rata kiri - :---: — Rata tengah - ---: — Rata kanan

3.4.3 Penulisan Matematika

Matematika inline ditulis dengan tanda dollar tunggal:

Rumus $E = mc^2$ adalah rumus relativitas.

Matematika display ditulis dengan tanda dollar ganda:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

Pandoc akan meneruskan sintaks LaTeX ini langsung ke template LaTeX tanpa perubahan.

3.4.4 Penulisan Kode

Kode inline ditulis dengan backtick tunggal:

Gunakan perintah `'pandoc --help'` untuk melihat bantuan.

Blok kode ditulis dengan triple backtick dan dapat menyertakan bahasa untuk syntax highlighting:

```
'''bash
pandoc input.md -o output.pdf --pdf-engine=pdflatex

### Referensi Gambar
```

Gambar direferensikan dengan path relatif terhadap direktori kerja Pandoc:

```
'''markdown
![Caption Gambar](gambar/nama-file.png)
```

Pandoc akan menghasilkan lingkungan figure LaTeX secara otomatis. Caption akan muncul di bawah gambar dengan format “Gambar 4.1” sesuai pengaturan template.

3.5 Studi Kasus: Project Laporan Generator

Project Laporan Generator adalah implementasi nyata dari pipeline otomatisasi dokumen yang dibahas dalam laporan ini. Project ini berisi seluruh komponen pipeline:

1. **File konten:** `cover.md`, `chapters/bab*.md`, `daftar-pustaka.md`
2. **File metadata:** `metadata.yml`, `references.bib`
3. **File template:** `template.latex`
4. **Skrip build:** `build.sh`
5. **File pendukung:** `logo.jpg`

Semua file berada dalam satu direktori yang merupakan root dari project. Struktur ini sengaja dibuat sederhana untuk memudahkan penggunaan dan modifikasi.

Untuk menghasilkan PDF, pengguna cukup menjalankan:

```
cd ~/Projects/laporan-generator
chmod +x build.sh
./build.sh
```

Skrip akan secara otomatis: 1. Membaca file-file sumber 2. Memproses gambar (jika ada) 3. Menjalankan Pandoc dengan template LaTeX 4. Menghasilkan file `Laporan.pdf`

Seluruh proses berlangsung dalam hitungan detik dan menghasilkan dokumen dengan format yang konsisten dan profesional.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Struktur Direktori

Implementasi pipeline menghasilkan struktur direktori sebagai berikut:

```
laporan-generator/  
|-- cover.md           # Halaman sampul  
|-- template.latex     # Template LaTeX  
|-- build.sh           # Skrip build  
|-- metadata.yml       # Judul, penulis, institusi  
|-- references.bib     # Daftar pustaka (BibTeX)  
|-- chapters/         # BAB 1-5 (modular)  
|   |-- bab1-pendahuluan.md  
|   |-- bab2-tinjauan-pustaka.md  
|   |-- bab3-metodologi.md  
|   |-- bab4-hasil-dan-pembahasan.md  
|   \-- bab5-penutup.md  
|-- logo.jpg           # Logo institusi  
|-- daftar-pustaka.md  # Fallback heading daftar pustaka  
|-- Makefile           # Target build, watch, test  
|-- test.sh            # Test suite  
|-- README.md          # Dokumentasi  
|-- LICENSE            # Lisensi MIT  
|-- .gitignore         # Ignore rules
```

Fungsi masing-masing file:

cover.md — Berisi halaman sampul laporan yang ditulis dalam format LaTeX. File ini disisipkan ke dalam dokumen melalui flag `--include-before-body`. Cover mencakup judul, logo institusi, identitas penyusun, dan informasi institusi. Selain cover, file ini juga berisi kata pengantar yang ditulis dalam format LaTeX.

template.latex — Template LaTeX yang mendefinisikan format dan layout dokumen. Template ini mencakup seluruh pengaturan yang diperlukan: document class, font, margin, header, footer, penomoran, daftar isi, dan styling. Template menggunakan variabel Pandoc (`$body$`, `$for(include-before)$`, dll.) untuk menyisipkan konten dari file Markdown.

build.sh — Skrip Bash yang mengorkestrasi seluruh pipeline. Skrip ini menangani persiapan file, pemrosesan gambar, eksekusi Pandoc, dan pembersihan. Semua operasi dilakukan secara otomatis tanpa intervensi manual.

logo.jpg — File gambar logo institusi yang ditampilkan pada halaman sampul. Logo dikonversi dari PNG ke JPG untuk mengurangi ukuran file dan meningkatkan kompatibilitas

dengan pdfLaTeX.

chapters/ — Direktori yang berisi file Markdown terpisah per bab (bab1 sampai bab5). Setiap file ditulis dalam format Pandoc Markdown. Pendekatan modular ini memudahkan kolaborasi dan version control karena perubahan per bab dapat dilacak secara independen.

daftar-pustaka.md — File yang berisi daftar pustaka atau referensi. File ini disisipkan setelah body dokumen melalui flag `--include-after-body`. Format penulisan mengikuti gaya APA.

4.2 Analisis Template LaTeX

Template LaTeX terdiri dari beberapa blok pengaturan yang masing-masing memiliki fungsi spesifik.

4.2.1 Document Class dan Font

```
\documentclass[12pt,a4paper,oneside]{book}

\usepackage[T1]{fontenc}
\pdfmapfile{+nimbus15.map}
\renewcommand{\rmdefault}{NimbusSerif}
\renewcommand{\familydefault}{\rmdefault}
```

Document class book dipilih karena menyediakan struktur chapter yang sesuai untuk laporan. Opsi 12pt menghasilkan ukuran font dasar 12 poin. Penggunaan `\pdfmapfile` dan `\renewcommand{\rmdefault}` mengaktifkan font Nimbus Serif sebagai font default.

4.2.2 Margin dan Spasi

```
\usepackage[margin=2.5cm,top=2cm,bottom=3cm,footskip=40pt]{geometry}
\usepackage{setspace}
\onehalfspacing
\setlength{\parindent}{1.5cm}
```

Margin kiri dan kanan 2,5 cm, atas 2 cm, bawah 3 cm. Spasi baris 1,5 dengan indentasi paragraf 1,5 cm. Pengaturan ini sesuai dengan standar laporan akademik yang umum digunakan di Indonesia.

4.2.3 Penomoran Bab dan Section

```
\renewcommand{\chaptername}{BAB}
\renewcommand{\thechapter}{\Roman{chapter}}
\renewcommand{\thesection}{\arabic{chapter}.\arabic{section}}
```

```
\renewcommand{\thesubsection}{\arabic{chapter}.\arabic{section}.\%
\arabic{subsection}}
```

Bab diberi nama “BAB” dengan angka Romawi, menghasilkan format seperti “BAB I” dan “BAB II”. Sub-bab menggunakan angka Arab dengan format “1.1” dan “2.3”. Sub-sub-bab menggunakan format “1.1.1”.

4.2.4 Format Chapter

```
\titleformat{\chapter}[display]
{\normalfont\bfseries\centering\fontsize{14}{18}\selectfont}
{\chaptername\ \thechapter}{10pt}{\fontsize{14}{18}\selectfont}
```

Chapter ditampilkan dengan format bold, centered, ukuran font 14pt dengan spasi 18pt. Nomor bab ditampilkan pada baris terpisah dari judul bab. Jarak antara nomor dan judul adalah 10pt.

4.3 Analisis Build Script

4.3.1 Setup dan Error Handling

```
set -e
TMPDIR=$(mktemp -d)
trap "rm -rf $TMPDIR" EXIT
```

Tiga baris ini merupakan fondasi keandalan script: - set -e menghentikan script saat terjadi error - mktemp -d membuat direktori temporer yang aman - trap memastikan direktori temporer dibersihkan, bahkan jika script gagal di tengah jalan

4.3.2 Proses Copy dan Preparasi

```
cp "$OUTDIR/cover.md" "$TMPDIR/"
cp "$OUTDIR/template.latex" "$TMPDIR/"
cp "$OUTDIR/logo.jpg" "$TMPDIR/"
cp "$OUTDIR/metadata.yml" "$TMPDIR/"
cp "$OUTDIR/references.bib" "$TMPDIR/"

if [ -d "$OUTDIR/chapters" ]; then
  cp "$OUTDIR/chapters"/*.md "$TMPDIR/"
else
  cp "$OUTDIR/isi-laporan.md" "$TMPDIR/"
fi
```

Semua file disalin ke direktori temporer untuk menjaga direktori kerja tetap bersih. File-file sisa kompilasi LaTeX (.aux, .log, .out, .toc) akan tertinggal di direktori temporer yang nantinya dihapus.

4.3.3 Eksekusi Pandoc

```
pandoc \  
  $INPUT_FILES \  
  --template="template.latex" \  
  --include-before-body="cover.md" \  
  --include-after-body="daftar-pustaka.md" \  
  --metadata-file="metadata.yml" \  
  --citeproc \  
  --bibliography="references.bib" \  
  --top-level-division=chapter \  
  --pdf-engine=pdflatex \  
  -o "$REPORT" 2>&1
```

Flag `--include-before-body` menyisipkan `cover.md` di awal dokumen (sebelum BAB 1). Flag `--include-after-body` menyisipkan `daftar-pustaka.md` di akhir dokumen (setelah BAB 5). Flag `--top-level-division=chapter` memastikan heading level 1 di Markdown dikonversi menjadi perintah `\chapter{}` di LaTeX, bukan `\section{}` atau `\part{}`.

4.3.4 Font Definition Generation

Script menghasilkan dua file yang diperlukan oleh font Nimbus Serif:

1. `fonttools_ts1.enc` — File encoding untuk TS1 (Text Companion Symbols) yang mendefinisikan pemetaan karakter simbol seperti tanda mata uang, tanda kurung, dan simbol tipografi lainnya.
2. `t1jtm.fd` — File font definition yang mendeklarasikan font shape untuk keluarga font JTM yang digunakan oleh Nimbus Serif. File ini mendefinisikan kombinasi weight (medium, bold, bx) dan shape (normal, italic, slanted, small caps) yang tersedia.

Kedua file ditulis langsung dari dalam script menggunakan heredoc, sehingga tidak memerlukan file eksternal.

4.4 Output Pipeline

4.4.1 File yang Dihasilkan

Pipeline menghasilkan satu file output utama: `Laporan.pdf`. File ini merupakan dokumen PDF yang siap digunakan dengan format sebagai berikut:

Aspek	Spesifikasi
Ukuran kertas	A4 (210 mm x 297 mm)
Font	Times New Roman (Nimbus Serif)
Ukuran font	12pt
Spasi baris	1,5 spasi
Margin kiri/kanan	2,5 cm
Margin atas/bawah	2 cm / 3 cm
Indentasi paragraf	1,5 cm
Penomoran bab	Romawi (BAB I, BAB II)
Penomoran sub-bab	Arab (1.1, 2.1)
Daftar isi	Otomatis
Lisensi font	Open source (GPL)

4.4.2 Perbandingan dengan Metode Konvensional

Aspek	Pipeline Otomatis	Word Processor
Format konten	Markdown (teks)	Binary (docx)
Version control	Git-friendly	Tidak kompatibel
Konsistensi format	Terjamin oleh template	Bergantung pengguna
Waktu revisi format	Ubah template, rebuild	Edit manual per halaman
Kolaborasi	Merge via Git	Track changes terbatas
Output	PDF langsung	Export manual
Dependensi	Pandoc + LaTeX	MS Word / LibreOffice

Pipeline otomatis unggul dalam konsistensi format, kemudahan revisi, dan kompatibilitas dengan version control. Sementara word processor unggul dalam kemudahan penggunaan awal (WYSIWYG) dan tidak memerlukan instalasi tools tambahan.

4.5 Version Control dengan Git

4.5.1 Keunggulan File Teks

Penggunaan Markdown sebagai format konten memberikan keunggulan signifikan dalam version control. File Markdown adalah teks biasa, sehingga Git dapat melacak perubahan secara granular, baris per baris.

Perubahan yang sebelumnya sulit dilacak di file binary DOCX: - Penambahan satu kalimat → diff satu baris - Perbaikan typo → diff beberapa karakter - Perubahan format tabel →

diff struktur tabel

4.5.2 Workflow Kolaborasi

Dengan pipeline otomatis, workflow kolaborasi menjadi:

1. Setiap kontributor menulis konten di file Markdown
2. Perubahan di-commit ke Git
3. Merge dilakukan dengan Git standar
4. PDF di-build ulang dengan `./build.sh`
5. Output PDF selalu sinkron dengan konten terbaru

4.5.3 .gitignore

File `.gitignore` dikonfigurasi untuk mengabaikan file-file yang tidak perlu di-version control:

```
*.pdf
*.aux
*.log
*.out
*.toc
*.lof
*.lot
*.fls
*.fdb_latexmk
```

File PDF tidak perlu di-commit karena dapat dihasilkan kapan saja dengan menjalankan `build.sh`. File sisa kompilasi LaTeX (aux, log, out, toc) juga diabaikan karena bersifat sementara.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan pembahasan yang telah dilakukan, dapat ditarik kesimpulan sebagai berikut:

1. **Markdown, LaTeX, dan Pandoc** memiliki peran yang saling melengkapi dalam otomatisasi dokumen. Markdown berfungsi sebagai format penulisan konten yang ringan dan mudah dibaca. LaTeX berfungsi sebagai sistem typesetting untuk format dokumen profesional. Pandoc berfungsi sebagai jembatan konversi yang menghubungkan keduanya.
2. **Template LaTeX** untuk laporan akademik terdiri dari beberapa komponen utama: document class (book), font (Nimbus Serif), margin (2,5 cm kiri/kanan, 2 cm atas, 3 cm bawah), spasi (1,5 spasi), penomoran (Romawi untuk BAB, Arab untuk sub-bab), dan daftar isi (dengan entri BAB tebal).
3. **Alur konversi dokumen** dari Markdown ke PDF melalui tiga tahap: parsing Markdown ke AST oleh Pandoc, konversi AST ke LaTeX menggunakan template, dan kompilasi LaTeX ke PDF oleh pdfLaTeX. Seluruh alur dijalankan dengan satu perintah Pandoc.
4. **Penanganan gambar** dalam pipeline menggunakan ImageMagick untuk menghapus alpha channel dari file PNG, mencegah masalah kompatibilitas dengan pdfLaTeX. Perintah `convert -alpha off` mengganti area transparan dengan latar belakang putih.
5. **Skrip Bash** berhasil mengotomatisasi seluruh pipeline dengan fitur-fitur penting: error handling dengan `set -e`, direktori temporer dengan `mktemp`, pembersihan otomatis dengan `trap`, dan eksekusi Pandoc dengan parameter yang tepat.
6. **Pipeline otomatisasi** yang diimplementasikan pada project Laporan Generator berhasil menghasilkan dokumen PDF dengan format konsisten dan profesional. Struktur file yang modular (direktori `chapters/`, `metadata.yml`, `references.bib`, dan `Makefile`) memudahkan penggunaan dan modifikasi.

5.2 Saran

Untuk pengembangan pipeline otomatisasi dokumen selanjutnya, beberapa saran yang dapat diberikan:

1. **Continuous Integration / Continuous Deployment (CI/CD)** — Pipeline dapat diintegrasikan dengan GitHub Actions atau GitLab CI untuk build otomatis setiap kali ada perubahan di repository. Setiap push akan menghasilkan PDF terbaru tanpa perlu menjalankan build manual.
2. **Multi-format output** — Pandoc mendukung berbagai format output selain PDF, seperti HTML, EPUB, dan DOCX. Pipeline dapat dikembangkan untuk menghasilkan beberapa format sekaligus.
3. **Template kustomisasi** — Template LaTeX dapat dikembangkan dengan lebih banyak variasi, seperti template untuk laporan magang, skripsi, makalah, dan jurnal. Masing-masing template dapat memiliki pengaturan format yang berbeda.
4. **Integrasi dengan reference manager** — Daftar pustaka dapat diintegrasikan dengan file BibTeX atau CSL (Citation Style Language) untuk manajemen referensi yang lebih baik.
5. **Penjadwalan build otomatis** — Pipeline dapat dijadwalkan menggunakan cron job untuk build periodik, berguna untuk dokumen yang memerlukan update rutin.
6. **Validasi konten** — Pipeline dapat dilengkapi dengan validasi otomatis untuk memeriksa kesalahan penulisan (typo, inkonsistensi format, referensi yang tidak lengkap) sebelum build.

DAFTAR PUSTAKA

- Foundation, F. S. (2024). *GNU bash reference manual*. <https://www.gnu.org/software/bash/manual/>
- Gruber, J. (2004). *Markdown*. <https://daringfireball.net/projects/markdown/>
- Knuth, D. E. (1984). *The TeXbook*. Addison-Wesley.
- Lamport, L. (1994). *LaTeX: A document preparation system* (2nd ed.). Addison-Wesley.
- LLC, I. S. (2024). *ImageMagick documentation*. <https://imagemagick.org/script/documentation.php>
- Love, R. (2010). *Linux kernel development* (3rd ed.). Addison-Wesley.
- MacFarlane, J. (2024). *Pandoc user's guide*. <https://pandoc.org/MANUAL.html>
- Welsh, M., Dalheimer, M. K., & Kaufman, L. (2019). *Running linux* (6th ed.). O'Reilly Media.